Image Super-Resolution System

Complete Technical Documentation

Application Architecture | API Reference | Deployment | Integration Guide

---

Live API Endpoint:
`https://image-super-resolution-app.onrender.com`

Version 2.0 | March 2026

- by Arkaprava Roy

1. Project Overview

This document provides complete technical documentation for the Image Super-Resolution System — an AI-powered image upscaling service built with PyTorch and FastAPI. The system accepts low-resolution images and produces high-quality upscaled outputs at 2x or 4x resolution using the EDSR (Enhanced Deep Super-Resolution) deep learning model.

The system is composed of two independent components:

Component	File	Description
Local Application	app.py	Streamlit-based web UI for local execution
REST API Server	api.py	FastAPI backend exposing HTTP endpoints for remote integration

> **Architecture Note:** The local application (app.py) and the API server (api.py) share identical inference logic but serve different consumers. The local application is intended for standalone use. The API server exposes the same functionality over HTTP, enabling integration with any external client — web frontends, mobile applications, or automated pipelines.

2. Repository Structure

The repository is organized as follows:

```
image-super-resolution/
  app.py                          # Streamlit local application
  api.py                          # FastAPI REST API server
  requirements.txt                # Python package dependencies
  README.md                       # Repository overview
  API.md                          # API endpoint reference
  Image Upscalar Documentation.pdf  # Original project documentation
```

3. app.py — Local Streamlit Application

The local application provides a browser-based graphical interface for direct use on the host machine. It integrates model loading, inference, image comparison, and file download into a single self-contained module.

3.1 Execution

```
# Launch the local application from the project directory:
streamlit run app.py

# Interface is accessible at:
# http://localhost:8501
```

3.2 Annotated Source Code

```
import os
# Directs PyTorch to store downloaded model weights in /tmp/torch.
# Prevents permission errors in restricted or cloud environments
# where the default home directory may not be writable.
os.environ['TORCH_HOME'] = '/tmp/torch'

import streamlit as st            # Web UI framework
import torch                       # PyTorch deep learning library
from torchsr.models import edsr   # EDSR super-resolution model
from PIL import Image             # Image I/O and conversion (Pillow)
import torchvision.transforms as T  # Tensor/image transform utilities
from streamlit_image_comparison import image_comparison  # Before/after slider
import io                         # In-memory byte stream handling
```

```
# ── PAGE CONFIGURATION ──────────────────────────────────────
st.set_page_config(
    page_title='Image Super-Resolution',
    layout='centered'
)
st.title('Image Super-Resolution App')
st.write('Upscale images using deep learning (EDSR)')

# Select compute device: GPU (cuda) if available, otherwise CPU.
# GPU inference is significantly faster for large images.
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
# ── MODEL LOADING ───────────────────────────────────────────
# @st.cache_resource: Streamlit decorator that loads the models exactly
# once per session and caches them in memory. Without this decorator,
# the models would reload on every UI interaction, causing ~30s delays.
@st.cache_resource
def load_models():
    return {
        2: edsr(scale=2, pretrained=True).to(device).eval(),
        # scale=2      -> model variant trained for 2x upscaling
        # pretrained=True -> load official pre-trained weights
```

```
        # .to(device)  -> transfer model parameters to GPU or CPU
        # .eval()      -> disable dropout/batch-norm training behaviour
        4: edsr(scale=4, pretrained=True).to(device).eval(),
        # Identical configuration for the 4x upscaling variant
    }

# models is a dict: { 2: <EDSR_2x_model>, 4: <EDSR_4x_model> }
models = load_models()
```

```
# ── USER INTERFACE ──────────────────────────────────────────
# File uploader widget — restricts input to supported image formats
uploaded_file = st.file_uploader(
    'Upload an image',
    type=['png', 'jpg', 'jpeg']
)

# Radio button for scale factor selection
scale = st.radio(
    'Select upscale factor',
    [2, 4],
    horizontal=True
)
```

```
# ── INFERENCE PIPELINE ──────────────────────────────────────
if uploaded_file is not None:

    # Open uploaded file as a PIL Image.
    # .convert('RGB') normalises the colour space —
    # removes alpha channel (RGBA) and converts greyscale to RGB.
    img = Image.open(uploaded_file).convert('RGB')
    st.subheader('Original Image')
    st.image(img, use_container_width=True)

    to_tensor = T.ToTensor()   # PIL Image -> float32 tensor in range [0.0, 1.0]
    to_pil    = T.ToPILImage() # float32 tensor -> PIL Image

    # .unsqueeze(0): inserts a batch dimension at position 0.
    # Shape transforms: [C, H, W] -> [1, C, H, W]
    # The model expects a 4-dimensional input (batch, channels, height, width).
    lr = to_tensor(img).unsqueeze(0).to(device)

    # Retrieve the pre-loaded model for the selected scale factor
    model = models[scale]

    # torch.no_grad(): disables gradient computation graph construction.
    # Not required during inference — disabling it reduces memory usage
    # and increases execution speed.
    with torch.no_grad():
        sr = model(lr)   # Forward pass — produces super-resolved tensor
```

```
    # Post-processing:
    # .squeeze(0)    -> removes batch dimension: [1,C,H,W] -> [C,H,W]
    # .clamp(0, 1)   -> clips values outside [0,1] to prevent artefacts
    # .cpu()         -> transfers tensor back to CPU for PIL conversion
    sr_img = to_pil(sr.squeeze(0).clamp(0, 1).cpu())
```

```
# ── OUTPUT AND DOWNLOAD ──────────────────────────────────────────

    # Render interactive before/after comparison slider
    st.subheader('Before / After Comparison')
    image_comparison(
        img1=img,
        img2=sr_img,
        label1='Before (Low Resolution)',
        label2=f'After ({scale}x Super-Resolution)'
    )

    # Serialise the output image to an in-memory PNG byte buffer.
    # BytesIO avoids writing to disk — the bytes are served directly
    # to the download button in the browser.
    buf = io.BytesIO()
    sr_img.save(buf, format='PNG')
    byte_im = buf.getvalue()

    st.download_button(
        label='Download Super-Resolved Image',
        data=byte_im,
        file_name=f'SR_x{scale}.png',
        mime='image/png'
    )

    # Display resolution metadata
    st.caption(
        f'Input size: {img.size[0]}x{img.size[1]} | '
        f'Output size: {sr_img.size[0]}x{sr_img.size[1]}'
    )
```

4. api.py — FastAPI REST Server

The API server exposes the super-resolution inference pipeline over HTTP. It is designed to be consumed by external clients — web applications, mobile clients, or automated pipelines — without requiring any knowledge of the underlying model architecture.

4.1 Local Execution

```
# Install API-specific dependencies (one time):
pip install fastapi uvicorn python-multipart

# Start the development server with hot-reload:
uvicorn api:app --reload

# Interactive API documentation is available at:
# http://127.0.0.1:8000/docs
```

4.2 Annotated Source Code

```
import os
os.environ['TORCH_HOME'] = '/tmp/torch'

from fastapi import FastAPI, UploadFile, File, HTTPException
# FastAPI       -> ASGI web framework for building REST APIs
# UploadFile    -> type annotation for multipart file uploads
# File          -> FastAPI dependency marker for file parameters
# HTTPException -> raises HTTP error responses with status codes

from fastapi.middleware.cors import CORSMiddleware
# CORS (Cross-Origin Resource Sharing) middleware.
# Web browsers enforce the Same-Origin Policy: a page at domain A
# cannot call an API at domain B unless the API explicitly permits it.
# This middleware adds the required headers to all responses,
# allowing any origin to call this API.

from fastapi.responses import StreamingResponse
# Streams binary data (the PNG image) directly to the client
# without buffering the entire response in memory first.

import torch
from torchsr.models import edsr
from PIL import Image
import torchvision.transforms as T
import io
```

```
# ── APPLICATION INITIALISATION ──────────────────────────────────────
app = FastAPI()   # Instantiate the FastAPI application

# CORS configuration: allow_origins=['*'] permits all origins.
# For production deployments, this should be restricted to the
# specific domains of authorised client applications.
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*']
)

device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

```
# ── LAZY MODEL LOADING ──────────────────────────────────────────────
# Models are loaded on first request rather than at server startup.
# Rationale: loading both EDSR models simultaneously (~400MB total)
# exceeds the available RAM on free-tier hosting (512MB limit).
# Lazy loading ensures only the requested scale model is in memory.

models = {}   # Model registry: populated on demand

def get_model(scale):
    if scale not in models:
        # First call for this scale: load and cache the model
        models[scale] = edsr(scale=scale, pretrained=True).to(device).eval()
    return models[scale]

# Behaviour:
# - First request with scale=2 -> loads EDSR 2x model into models[2]
# - Subsequent requests with scale=2 -> returns cached models[2] instantly
# - scale=4 model is only loaded if a 4x request is received
```

```
# ── HEALTH CHECK ENDPOINT ───────────────────────────────────────────
# GET /health
# Purpose: allows client applications and infrastructure monitoring
# tools to verify the service is operational before sending requests.
@app.get('/health')
def health():
    return {'status': 'ok'}

# Response: HTTP 200
# Body: { 'status': 'ok' }
```

```
# ── UPSCALE ENDPOINT ────────────────────────────────────────────────
# POST /upscale
# Accepts a multipart image upload and a scale query parameter.
```

```
# Returns the super-resolved image as a PNG byte stream.
@app.post('/upscale')
async def upscale(file: UploadFile = File(...), scale: int = 2):
    # file: UploadFile  -> the uploaded image (required)
    # scale: int = 2    -> upscale factor; defaults to 2 if omitted

    # Input validation — scale factor
    if scale not in (2, 4):
        raise HTTPException(status_code=400, detail='scale must be 2 or 4')

    # Input validation — file type
    if not file.content_type.startswith('image/'):
        raise HTTPException(status_code=400, detail='File must be an image')

    # Read the raw bytes from the multipart upload.
    # 'await' is required because file.read() is an asynchronous operation.
    contents = await file.read()

    # Wrap bytes in BytesIO to provide a file-like interface for PIL.
    img = Image.open(io.BytesIO(contents)).convert('RGB')

    # ── Inference pipeline (identical to app.py) ──────────────────
    to_tensor = T.ToTensor()
    to_pil    = T.ToPILImage()
    lr        = to_tensor(img).unsqueeze(0).to(device)
    model     = get_model(scale)

    with torch.no_grad():
        sr = model(lr)

    sr_img = to_pil(sr.squeeze(0).clamp(0, 1).cpu())
    # ── End inference ─────────────────────────────────────────────

    # Serialise output to an in-memory PNG buffer
    buf = io.BytesIO()
    sr_img.save(buf, format='PNG')
    buf.seek(0)   # Reset stream position to beginning before reading

    # Return PNG bytes as a streaming HTTP response
    return StreamingResponse(buf, media_type='image/png')
```

5. Request Lifecycle

The following table describes the complete sequence of operations that occur when a client submits an upscale request to the API.

#	Layer	Operation
1	Client application	An image file is packaged as multipart/form-data and submitted via HTTP POST to /upscale with the desired scale parameter.
2	Network	The HTTP request is routed over the internet to the Render-hosted server at https://image-super-resolution-app.onrender.com.
3	FastAPI — Validation	The framework validates that the scale parameter is 2 or 4 and that the uploaded file has an image MIME type. Invalid requests receive an HTTP 400 response.
4	FastAPI — Preprocessing	File bytes are read asynchronously and opened as a PIL RGB image. The image is converted to a normalised float32 PyTorch tensor with a batch dimension added.
5	EDSR Model — Inference	The input tensor is passed through the EDSR neural network in no-grad mode. The model performs a forward pass and returns a super-resolved output tensor.
6	FastAPI — Postprocessing	The output tensor is clamped to [0, 1], transferred to CPU, and converted back to a PIL Image. The result is serialised to an in-memory PNG byte buffer.
7	FastAPI — Response	A StreamingResponse delivers the PNG bytes to the client with Content-Type: image/png and HTTP status 200.
8	Client application	The response body contains the raw PNG image data. The client application reads the binary content and renders or saves it as required.

6. API Reference

6.1 Base URL

```
https://image-super-resolution-app.onrender.com
```

An interactive API explorer (Swagger UI) is available at:

```
https://image-super-resolution-app.onrender.com/docs
```

6.2 Endpoints

GET /health

Returns the operational status of the service. Intended for health monitoring and connectivity verification.

Property	Value
Method	GET
Path	/health
Auth required	No
Success response	HTTP 200 — { "status": "ok" }

POST /upscale

Accepts a multipart image upload and returns a super-resolved PNG image at the specified scale factor.

Property	Value
Method	POST
Path	/upscale
Content-Type	multipart/form-data
Auth required	No
Success response	HTTP 200 — PNG image bytes (Content-Type: image/png)

Query Parameters

Parameter	Type	Required	Default	Description
scale	integer	No	2	Upscale factor. Accepted values: 2 or 4.

Request Body — multipart/form-data

Field	Type	Required	Description
file	File	Yes	The source image to be upscaled. Accepted MIME types: image/png, image/jpeg.

Error Responses

HTTP Status	Condition	Response body (detail field)
400	Invalid scale value	scale must be 2 or 4
400	Non-image file uploaded	File must be an image
422	Missing required field	Validation error — field: file, type: missing
500	Internal server error	Unhandled exception during inference

7. Dependencies

All dependencies are declared in requirements.txt. The table below describes the role of each package within the system.

Package	Role
torch	PyTorch deep learning framework — provides tensor operations, GPU support, and the inference engine for EDSR.
torchvision	Official PyTorch library for image transforms. Provides ToTensor and ToPILImage used in the preprocessing and postprocessing stages.
torchsr	Third-party library providing pre-trained super-resolution models. Exposes the edsr() factory function used to instantiate the 2x and 4x models.
Pillow	Python Imaging Library fork. Used for opening, colour-space conversion, and serialisation of image files throughout both application components.
streamlit	Rapid web UI framework used exclusively by app.py to build the local browser interface.
streamlit-image-comparison	Streamlit component providing the interactive drag-slider for before/after image comparison in the local application.
fastapi	High-performance ASGI web framework used to define and serve the REST API endpoints in api.py.
uvicorn	ASGI server that executes the FastAPI application. Handles HTTP connections, request parsing, and response delivery.
python-multipart	Required by FastAPI to parse multipart/form-data request bodies. Without this package, file upload endpoints do not function.

8. Deployment

8.1 Hosting Platform

The API server is deployed on Render (render.com) as a Python web service. Render monitors the connected GitHub repository and triggers an automatic redeploy on every new commit to the main branch.

8.2 Build and Start Process

On each deployment, Render executes the following sequence:

1. Install dependencies: pip install -r requirements.txt
2. Start the server: uvicorn api:app --host 0.0.0.0 --port 10000

8.3 Free Tier Constraints

Constraint	Detail
Idle shutdown	The service suspends after 15 minutes of inactivity. The first request after suspension incurs a cold start latency of approximately 50-60 seconds.
RAM limit	512 MB. Lazy model loading is employed specifically to stay within this constraint.
Compute	CPU only — no GPU acceleration on the free tier. Inference latency is higher than local GPU execution.
Build time limit	30 minutes per build. Render was selected over alternatives (e.g. Railway, 10-minute limit) specifically to accommodate the large PyTorch dependency.

8.4 Deploying an Update

```
# Stage all changed files
git add .

# Commit with a descriptive message
git commit -m 'describe the change'

# Push to the main branch — Render detects this and redeploys automatically
git push
```

9. Client Integration Guide

This section provides implementation examples for consuming the API from common client environments.

9.1 JavaScript (Browser / Node.js)

> **Note:** Do not set the Content-Type header manually when sending FormData. The browser sets it automatically with the correct multipart boundary string. Overriding it will cause the server to reject the request.

```
/**
 * Sends an image file to the super-resolution API and returns
 * an object URL pointing to the upscaled PNG.
 *
 * @param {File}   imageFile - The source image file object
 * @param {number} scale     - Upscale factor: 2 or 4
 * @returns {Promise<string>} Object URL of the upscaled image
 */
async function upscaleImage(imageFile, scale) {
  const BASE_URL = 'https://image-super-resolution-app.onrender.com';

  const formData = new FormData();
  formData.append('file', imageFile);  // field name must be 'file'

  const response = await fetch(
    `${BASE_URL}/upscale?scale=${scale}`,
    { method: 'POST', body: formData }
  );

  if (!response.ok) {
    const err = await response.json();
    throw new Error(err.detail || 'Upscaling request failed');
  }

  // The response body IS the image — not JSON.
  // Convert to Blob, then create a temporary browser URL.
  const blob = await response.blob();
  return URL.createObjectURL(blob);  // Assign to <img>.src to display
}

// Usage example:
const inputFile = document.getElementById('fileInput').files[0];
const upscaledUrl = await upscaleImage(inputFile, 2);
document.getElementById('resultImage').src = upscaledUrl;
```

9.2 Python

```
import requests

BASE_URL = 'https://image-super-resolution-app.onrender.com'

def upscale_image(source_path: str, scale: int = 2, output_path: str =
'upscaled.png'):
    """
    Sends a local image file to the super-resolution API.

    Args:
        source_path:  Path to the input image file.
        scale:        Upscale factor (2 or 4).
        output_path:  Destination path for the upscaled PNG.
    """
    with open(source_path, 'rb') as f:
        response = requests.post(
            f'{BASE_URL}/upscale',
            params={'scale': scale},
            files={'file': f}   # field name must be 'file'
        )

    response.raise_for_status()   # Raises HTTPError for 4xx/5xx responses

    with open(output_path, 'wb') as out:
        out.write(response.content)

    print(f'Saved upscaled image to: {output_path}')


# Example invocation:
upscale_image('input.jpg', scale=4, output_path='output_4x.png')
```

9.3 cURL

```
# 2x upscale — save result to upscaled_2x.png
curl -X POST 'https://image-super-resolution-app.onrender.com/upscale?scale=2' \
  -F 'file=@/path/to/image.jpg' \
  --output upscaled_2x.png

# 4x upscale — save result to upscaled_4x.png
curl -X POST 'https://image-super-resolution-app.onrender.com/upscale?scale=4' \
  -F 'file=@/path/to/image.jpg' \
  --output upscaled_4x.png

# Health check
curl https://image-super-resolution-app.onrender.com/health
```

9.4 Cold Start Handling

On the free hosting tier, the service suspends after a period of inactivity. Client applications should account for the cold start latency on the first request. The recommended approach is shown below:

```
// Recommended: pre-warm the service before the user initiates an upload.
// Call /health as soon as the page loads — this wakes the server
// so it is ready by the time the user selects an image.

async function warmUpServer() {
  try {
    await fetch('https://image-super-resolution-app.onrender.com/health');
  } catch (_) {
    // Ignore errors — this is a best-effort warm-up only
  }
}

// Call on page load:
window.addEventListener('load', warmUpServer);
```

10. Quick Reference

Item	Value
Live API base URL	https://image-super-resolution-app.onrender.com
Interactive documentation	https://image-super-resolution-app.onrender.com/docs
Health check endpoint	GET /health
Upscale endpoint	POST /upscale
Accepted scale values	2, 4
Request format	multipart/form-data — field name: file
Accepted image formats	PNG, JPG, JPEG
Response format	PNG image bytes — Content-Type: image/png
Cold start latency	~50-60 seconds after idle period
Start API server (local)	uvicorn api:app --reload
Start local application	streamlit run app.py
Install dependencies	pip install -r requirements.txt

-------End of Document-------